COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Corro, Yazinel Obias, Jave Joaquin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 **[Parallel Algorithms and Multithreading]**

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Screenshot (Code and Output)

```cpp
#include <iostream>
#include <thread>
#include <string>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
return 0;
}

```

```
Printing integer: 10
Printing string: T.I.P.
PS C:\Users\TIPQC\Downloads\HOA13>
```

Analysis

Corro	In this code, this demonstrates how to create a basic thread using integer and string. The command “t1.join’ is used to make the main program wait for t1 to

	complete its work before finishing the execution.
Obias	In this code, it makes a single thread called t1 and executes the function print passing the integer 10 with the string "T.I.P". The t1.join() will make the main thread complete before it ends the code.

Table 13-1. Simple One-Threaded Example

Part 2: Multi-Threaded Example

Screenshot (Code and Output)

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}
int main() {
    std::vector<std::string> s = {
    "T.I.P.",
    "Competent",
    "Computer",
    "Engineers"
    };
    std::vector<std::thread> threads;
    for (int i = 0; i < s.size(); i++) {
    threads.push_back(std::thread(print, i, s[i]));
    }
    for (auto &th : threads) {
    th.join();
    }
return 0;
}
```

```
3 : Engineers1 : Competent
2 : Computer
0 : T.I.P.
```

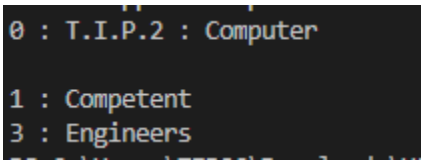
<table>
<tr><td colspan="2"> 
</td></tr>
<tr><td colspan="2">Analysis</td></tr>
<tr><td>Corro</td><td>This program demonstrates multithreading or multiple threads. The 6-9 line is the function of the code and the main includes the vector, string, etc. A loop uses “th.join()” to pause the main execution until a background thread finishes executing. And when the executing finishes it will output the message in numerical order. Also, when you run the program again, the output will change.</td></tr>
<tr><td>Obias</td><td>In this code, multithreading is used to print out a vector of strings along with their corresponding index numbers. The threads run concurrently, they finish at different times and can cause the console output to print out of sequence. Also, since several threads are attempting to display text simultaneously, the terminal output can easily get overlapping characters, mixed-up lines, or jumbled text.</td></tr>
</table>

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

<table>
<tr><td colspan="2">1. Write a definition of multithreading and its advantages/disadvantages.</td></tr>
<tr><td>Corro</td><td>Multithreading is a programming technique that lets one program perform multiple smaller tasks , called threads , simultaneously . The biggest upside of this approach is that it makes applications faster, more efficient, and much more responsive to the user. But it also has disadvantages, such as making the source code much harder to write, test and debug. Plus it can cause tricky bugs like data races when several threads try to change the same data at the same time.</td></tr>
<tr><td>Obias</td><td>Multithreading is a technique that lets a program execute multiple tasks concurrently to boost performance and keep applications responsive, though</td></tr>
</table>

	it increases code complexity and can cause concurrency issues like out-of-order execution, jumbled terminal output, or corrupted shared data.
2. Rationalize the use of multithreading by providing at least 3 use-cases.	
Corro	In modern software , multithreading is very useful . It allows various background operations to occur without freezing the main application . A good example is web browsers . One thread loads a webpage, and another downloads a file in the background. Another common use-case is in video games , which use multiple threads to handle graphics , audio and game physics all at the same time .
Obias	Multithreading is justified whenever an application must handle heavy or concurrent tasks—such as keeping a GUI smooth during background operations, processing simultaneous web server client requests, or speeding up massive computations across multiple CPU cores—without freezing or slowing down the system.
3. Differentiate between parallelism and concurrency.	
Corro	Concurrency is about doing many things at once . The way this works is that you rapidly switch between them on a single processor core , so it appears like they are happening together . In contrast, parallelism means that multiple tasks are executed at the exact same physical moment, and requires hardware that has multiple CPU cores. To wrap up: concurrency is about handling tasks and parallelism is about doing things simultaneously.
Obias	Concurrency is about dealing with multiple tasks at once by rapidly switching between them on a single core, whereas parallelism is about actually executing multiple tasks at the exact same instant using separate CPU cores.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

Create a global variable of type integer.
Create an add function that will take an integer parameter and add that value to the global variable.
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.

Display the value of global variables at different combinations of using the join() per thread. Such that:
o Display
o T1.join()
o Display
o T2.join()
o Display
o T3.join()
Provide an analysis based on the outputs of the multi-threading exercise.

Code

```cpp
#include <iostream>
#include <thread>
#include <mutex>
int globalVariable = 0;
std::mutex mtx;

// Function that adds an integer to the global variable
void add(int value)
{
    std::lock_guard<std::mutex> lock(mtx);
    globalVariable += value;
}

int main()
{
    // Create three threads
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    // Display the initial/current value
    std::cout << "Initial Display: " << globalVariable << std::endl;

    // Wait for Thread 1
    T1.join();
    std::cout << "After T1.join(): " << globalVariable << std::endl;

    // Wait for Thread 2
    T2.join();
    std::cout << "After T2.join(): " << globalVariable << std::endl;

    // Wait for Thread 3
    T3.join();
    std::cout << "After T3.join(): " << globalVariable << std::endl;

    return 0;
```

Output

```
Initial Display: 0
After T1.join(): 60
After T2.join(): 60
After T3.join(): 60
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-hoa1\HOA13P1>
```

Analysis	
Corro	We used multithreading to create three threads (T1,T2,T3) and to add different integer values to a shared global variable. Each thread adds its own value of 10, 20 and 30 respectively. We protected the global variable with a mutex so that multiple threads could not modify it at the same time.
Obias	From the output we can see that the final value of the global variable is 60 after the execution of all the three threads. Joining() causes the main program to wait for a specific thread to complete . Other threads can still run before their join() is called . Therefore the values displayed may be different depending on the execution order. This activity shows how multithreading works and why synchronization is important when multiple threads share the same variable.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Code

```
10  #include "selectionAlgo.h"
11  #include "../HOA7P1/supple/selectionAlgo.h"
12
13  using Clock = std::chrono::steady_clock;
14
15  double ms(Clock::time_point a, Clock::time_point b)
16  {
17      return std::chrono::duration<double, std::milli>(b - a).count();
18  }
19
20  // Sort chunks [start,end) in parallel, then merge pairwise
21  void parallelSelectionSort(std::vector<int>& v, int numThreads)
22  {
23      int n = v.size();
24      std::vector<int> bounds(numThreads + 1);
25      for (int t = 0; t <= numThreads; t++)
26          bounds[t] = (int)((long long)n * t / numThreads);
27
28      std::vector<std::thread> threads;
29      for (int t = 0; t < numThreads; t++)
30      {
31          threads.emplace_back([&v, &bounds, t]() {
32              selectionSort(v.data() + bounds[t], bounds[t + 1] - bounds[t]);
33          });
34      }
35      for (auto& th : threads) th.join();
36
37      // Merge sorted chunks (sequential, O(n) per merge)
38      for (int t = 1; t < numThreads; t++)
39          std::inplace_merge(v.begin(), v.begin() + bounds[t], v.begin() + bounds[t + 1]);
40  }
41
42  // Each thread counts its own slice into a private array (no data race)
43  void parallelCount(const std::vector<int>& v, int numThreads, int counts[6])
44  {
```

```
    int n = v.size();
    std::vector<std::vector<int>> local(numThreads, std::vector<int>(6, 0));
    std::vector<std::thread> threads;
    for (int t = 0; t < numThreads; t++)
    {
        int s = (int)((long long)n * t / numThreads);
        int e = (int)((long long)n * (t + 1) / numThreads);
        threads.emplace_back([&v, &local, t, s, e]() {
            for (int i = s; i < e; i++) local[t][v[i]]++;
        });
    }
    for (auto& th : threads) th.join();
    for (int t = 0; t < numThreads; t++)
        for (int c = 1; c <= 5; c++) counts[c] += local[t][c];
}

void runTest(int n, const std::vector<int>& threadCounts)
{
    std::vector<int> original(n);
    for (int i = 0; i < n; i++) original[i] = (std::rand() % 5) + 1;

    std::cout << "\n=== N = " << n << " votes ===\n";
    std::cout << std::left << std::setw(10) << "Threads"
              << std::setw(14) << "Sort (ms)"
              << std::setw(14) << "Count (ms)"
              << std::setw(10) << "Speedup"
              << "Correct?\n";

    double baseline = 0;
    std::vector<int> reference = original;
    std::sort(reference.begin(), reference.end());

```

```
80
81          auto t0 = Clock::now();
82          if (T == 1) selectionSort(v.data(), n);
83          else        parallelSelectionSort(v, T);
84          auto t1 = Clock::now();
85
86          int counts[6] = {0};
87          auto t2 = Clock::now();
88          parallelCount(v, T, counts);
89          auto t3 = Clock::now();
90
91          double sortMs = ms(t0, t1);
92          if (T == 1) baseline = sortMs;
93          bool ok = (v == reference) && (counts[1]+counts[2]+counts[3]+counts[4]+counts[5] == n);
94
95          std::cout << std::left << std::setw(10) << T
96                    << std::setw(14) << std::fixed << std::setprecision(3) << sortMs
97                    << std::setw(14) << ms(t2, t3)
98                    << std::setw(10) << std::setprecision(2) << (baseline / sortMs)
99                    << (ok ? "Yes" : "NO") << "\n";
```

```
86          int counts[6] = {0};
87          auto t2 = Clock::now();
88          parallelCount(v, T, counts);
89          auto t3 = Clock::now();
90
91          double sortMs = ms(t0, t1);
92          if (T == 1) baseline = sortMs;
93          bool ok = (v == reference) && (counts[1]+counts[2]+counts[3]+counts[4]+counts[5] == n);
94
95          std::cout << std::left << std::setw(10) << T
96                    << std::setw(14) << std::fixed << std::setprecision(3) << sortMs
97                    << std::setw(14) << ms(t2, t3)
98                    << std::setw(10) << std::setprecision(2) << (baseline / sortMs)
99                    << (ok ? "Yes" : "NO") << "\n";
100     }
101 }
102
103 int main()
104 {
105     std::srand(std::time(0));
106     std::cout << "Hardware threads available: "
107               << std::thread::hardware_concurrency() << "\n";
108
109     std::vector<int> tc = {1, 2, 4, 8};
110     runTest(101, tc);      // original problem size
111     runTest(20000, tc);    // larger size so threading effects are visible
112     runTest(50000, tc);
113     return 0;
114 }
```

Output

```
=== N = 101 votes ===
Threads   Sort (ms)     Count (ms)    Speedup   Correct?
1         0.010         1.887         1.00      Yes
2         0.865         0.776         0.01      Yes
4         0.897         1.234         0.01      Yes
8         0.867         0.701         0.01      Yes

=== N = 20000 votes ===
Threads   Sort (ms)     Count (ms)    Speedup   Correct?
1         231.846       0.728         1.00      Yes
2         60.196        0.522         3.85      Yes
4         20.095        0.677         11.54     Yes
8         8.397         1.147         27.61     Yes

=== N = 50000 votes ===
Threads   Sort (ms)     Count (ms)    Speedup   Correct?
1         1418.805      0.901         1.00      Yes
2         407.394       0.627         3.48      Yes
4         127.664       0.708         11.11     Yes
8         39.505        0.771         35.91     Yes
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-hoa1\HOA13P1>
```

Analysis	
Corro	I used the selection sort algorithm from my previous exercise in this code, but I made it multi-threaded by dividing the vote array into equal parts, each of which is sorted by a different thread. There are no race conditions because each thread operates on a separate slice of the array, and before combining the slices and tallying the votes, the main thread calls.join() on every thread. Since the sort only takes a few microseconds and the creation of threads is more expensive than the sort itself, adding threads slowed down the program with the initial 101 votes.
Obias	We used the Selection Sort code from our previous activity into a multi-threaded version by breaking the

	vote array into equal slices, letting each thread sort its assigned segment independently without any data races. For the small vote test, adding threads actually made the program run slower because thread creation overhead took longer than the brief sort itself. However, once we scaled up to the largest dataset, multi-threading drastically improved performance, dropping execution time from over a second on a single thread down to under forty milliseconds on eight threads for a massive speedup.

Conclusion	
Corro	I have learned that multithreading is to allow a program to do multiple tasks at the same time by creating threads with thread, and that join() makes the main program wait for a thread to finish. The procedure demonstrates that the order of the output of threads varies from run to run, because the order in which threads complete is not guaranteed. In part B, I learned that a shared variable has to be protected with a mutex, otherwise the threads can modify at the same time and give a wrong result.
Obias	I learned that multithreading improves performance and responsiveness, but adds complexity and can lead to problems such as jumbled output and corrupted shared data. The process and Part A helped me to understand the difference between concurrency (task switching) and parallelism (running multiple tasks at the same time on different cores). In Part B, the final value of 60 demonstrates that join() controls when the main thread waits, and a mutex keeps the correct result when threads access a shared variable.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.